Stichwortzettel: Technische Umsetzung Medienserver

Einstieg: „Was verkaufen wir euch?“

Wir verkaufen keine App und keinen Internetdienst. Wir verkaufen einen kleinen Computer, der im Schulnetz steht, mit unserer Videoplattform darauf. Eingerichtet, mit Anleitung übergeben.

Kurz gesagt: ein eigenes YouTube für die Schule. Lokal, ohne Internet, ohne Konto.

1. Produkt und Nutzen

- Plattform für Schulungs- und Unterrichtsvideos im eigenen Netzwerk.
- Aufruf über den Browser, am PC, Tablet oder Handy. Keine App, keine Installation.
- Für Lehrende: Video hochladen, Titel, Beschreibung, Thema und Sprache eintragen. Den Rest macht das System.
- Für Schüler: Seite öffnen, suchen oder filtern, abspielen.

Warum das besser ist als die üblichen Wege:

1. Die Videos bleiben in der Schule. Nichts wird zu Google oder YouTube hochgeladen, keine Werbung, kein Tracking.
2. Es läuft nur im Schulnetz. Fällt die Internetleitung aus, funktioniert es weiter.
3. Keine laufenden Kosten, also keine Cloud- und keine Speicherabos.
4. Kein Benutzerkonto nötig. Wer im Schulnetz ist, kommt sofort rein.
5. Alle Videos an einer Stelle mit Suche, statt verstreut auf USB-Sticks und Laufwerken.

2. Programmierung

Alles selbst geschrieben, rund 470 Zeilen Python dazu die Seitenvorlagen.

- Startseite mit Volltextsuche über Titel, Beschreibung und Thema. Dazu Filter nach Thema und Sprache, die sich kombinieren lassen.
- Detailseite mit HTML5-Player. Zusätzlich zur Browser-Steuerung zwei eigene Tasten für 10 Sekunden vor und zurück.
- Upload und Verwaltung: Videos hinzufügen, bearbeiten und löschen, inklusive Datei und Vorschaubild.
- Beim Upload liest das Programm die Metadaten selbst aus der Datei: Dauer, Erscheinungsjahr, Codec, Auflösung, Bildrate, Bitrate und Tonspur.
- Das Vorschaubild schneidet es ebenfalls selbst aus dem ersten Bild des Videos.
- Geht beim Auslesen etwas schief, bricht der Upload ab und die angefangene Datei wird gelöscht. Es bleiben keine halben Dateien liegen.
- Sicherheit: CSRF-Schutz bei allen Formularen, bereinigte Dateinamen, damit niemand aus dem Upload-Ordner ausbrechen kann, dazu ein Größenlimit von 2 GB.

- 12 automatische Tests prüfen Speichern, Suche, Bearbeiten, Löschen, die Sicherheitsfunktionen und das Streaming.

Technologien:

Bereich	Technologie
Backend	Python und Flask
Frontend	HTML, CSS, JavaScript, ohne Framework
Datenhaltung	JSON-Dateien, eine Datei pro Video
Videoanalyse	FFmpeg und FFprobe
Betrieb	Gunicorn und Nginx in Docker-Containern

Drei Entscheidungen, die ich erklären kann:

- **JSON statt Datenbank.** Angefangen haben wir mit SQLite und sind bewusst umgestiegen. Bei ein paar hundert Videos bringt eine Datenbank nichts. So ist jede Videoinfo eine lesbare Textdatei, die man einfach sichern und kopieren kann, und bei einem Fehler sieht man sofort, was drinsteht.
- **Kein Umwandeln beim Abspielen.** Dafür ist der kleine Rechner zu schwach. Wir setzen fertige MP4-Dateien voraus, mit H.264 und AAC, weil die jeder Browser direkt abspielt. Umgewandelt wird vorher einmal am PC.
- **Videos werden stückweise ausgeliefert.** Der Server beantwortet Teilabrufe. Deshalb startet die Wiedergabe sofort und man kann vorspulen, ohne die ganze Datei zu laden.

3. Hardware

- Ein Raspberry Pi als Server. 64-Bit-ARM mit Raspberry Pi OS, etwa kreditkartengroß, rund 5 Watt, lautlos, läuft durchgehend.
- Der Pi speichert die Videos, liefert die Webseite aus, streamt die Videos und erzeugt Metadaten und Vorschaubilder.
- Feste Adresse im Schulnetz: `10.0.13.3`. Aufruf über `http://10.0.13.3`, Port 80, also ohne Portangabe.
- Darauf laufen zwei Container über Docker Compose. Nginx nimmt die Anfragen an und liefert Videos und Bilder direkt aus, Gunicorn mit Flask macht Katalog, Upload und Verwaltung.
- Die Container starten nach einem Stromausfall von selbst wieder. Ein Healthcheck prüft laufend, ob die Anwendung antwortet.
- Videos und Metadaten liegen außerhalb der Container. Ein Update überschreibt also keine Daten.
- Auf der Benutzerseite braucht es nichts Besonderes, ein Browser reicht.

4. Aktueller Stand

Das läuft (am 13.10.2026 am Notebook nachgeprüft):

- Die Anwendung läuft komplett: Katalog mit Suche und Filtern, Detailseite mit Player und den 10-Sekunden-Tasten, Verwaltung mit Bearbeiten und Löschen, Upload.
- Ein echtes Testvideo liegt drin und spielt im Browser ab. Automatisch erkannt wurden Dauer (11 Sekunden), Erscheinungsjahr 2021 aus der Datei, Codec H.264, Auflösung 3840x2160, 24 Bilder pro Sekunde und 5,6 Mbit/s. Das Vorschaubild hat das Programm selbst erzeugt.
- Alle 12 Tests laufen durch, in etwa 5 Sekunden. Dabei wird auch echtes Streaming geprüft: Der Server beantwortet Teilabrufe mit `206 Partial Content`, das ist die Voraussetzung fürs Vorspulen.
- Der Zugang zum Pi steht: Netzwerk, SSH, Dateiübertragung. Docker ist installiert und läuft, ein Testcontainer hat funktioniert.
- Für den Pi gibt es zwei beschriebene Installationswege: mit Docker Compose und ohne Docker über systemd und Nginx.

> Falls der Pi beim Termin nicht läuft, zeige ich es am Notebook. Es ist dieselbe Software, nur ein anderer Rechner: `python main.py` starten und `http://127.0.0.1:5000` aufrufen.

Daran arbeite ich gerade:

- Inbetriebnahme am Pi. Der Docker-Build läuft etwa 6 Minuten und bricht dann mit einem Speicher- und Dateisystemfehler ab (`SIGBUS`, Ein-/Ausgabefehler). Vermutlich ist der Pi mit dem Bauen überfordert oder die SD-Karte hat einen Defekt.
- Was wir probieren: das Image nicht am Pi bauen, sondern fertig übertragen, die SD-Karte neu aufsetzen oder auf SSD umstellen, sonst die Variante ohne Docker.

Offen:

- Der Ton ist noch nie geprüft worden, unser Testvideo hat gar keine Tonspur. Als Nächstes laden wir ein Video mit Ton hoch.
- Unser Testvideo ist in 4K und damit zu groß. Für den Betrieb bereiten wir Videos in 720p oder 1080p vor, das spart Speicher und Netzwerk.
- Test auf echten Geräten: Bild, Ton, Vorspulen und Vollbild auf Notebook, Tablet und Handy.
- Lasttest: wie viele Zuschauer gleichzeitig schafft der Pi.
- Speicher und Sicherung: wie viel Platz brauchen wir, und wie oft sichern wir Videos und Metadaten.
- Es gibt bisher keine Anmeldung, jeder im Netz darf hochladen und löschen. Das muss die Schule entscheiden. Einen Login für die Verwaltung können wir nachrüsten.

5. Antworten auf mögliche Rückfragen

„Was genau verkauft ihr uns, Hardware, Software oder beides?“
Beides zusammen: den eingerichteten Raspberry Pi mit unserer Software, dazu Dokumentation und Übergabe. Die Software allein läuft auch auf einem Rechner, den die Schule schon hat.

„Warum nicht einfach YouTube, Moodle oder OneDrive?“
Weil die Videos dann das Haus verlassen. Dazu kommt: keine Abhängigkeit von einem Anbieter, keine Werbung, keine laufenden Kosten, und es funktioniert ohne Internet. Anpassen können wir es auch selbst.

„Was kostet das im Betrieb?“
Praktisch nur Strom, rund 5 Watt, und einmal die Hardware. Lizenzen fallen keine an, alle verwendeten Bausteine sind quelloffen.

„Ist das sicher? Kann jeder alles löschen?“
Abgesichert sind Upload-Pfade, Dateinamen, Dateigröße und Formularangriffe. Eine Anmeldung gibt es bisher nicht, weil es nur im geschlossenen Schulnetz läuft. Nachrüsten können wir sie jederzeit.

„Wie viele Zuschauer schafft der Pi gleichzeitig?“
Das messen wir gerade, eine belastbare Zahl habe ich noch nicht. Die Videos gehen direkt über Nginx raus und werden nicht umgewandelt, das ist der sparsamste Weg. Eng wird es eher beim Netzwerk und bei der SD-Karte als beim Prozessor.

„Was passiert, wenn der Pi ausfällt oder der Strom weg ist?“
Die Dienste starten automatisch neu. Videos und Metadaten sind normale Dateien und werden gesichert, damit lässt sich der Server auch auf neuer Hardware schnell wiederherstellen.

„Welche Videoformate gehen?“
MP4 mit H.264 und AAC, das spielt jeder Browser direkt. Andere Formate werden vorher einmal am PC umgewandelt, den Befehl dafür haben wir dokumentiert.

„Wie viele Videos verkraftet das?“
Begrenzt ist das vom Speicherplatz, nicht von der Software. Für ein paar hundert Videos ist die Lösung mit den JSON-Dateien schnell genug.